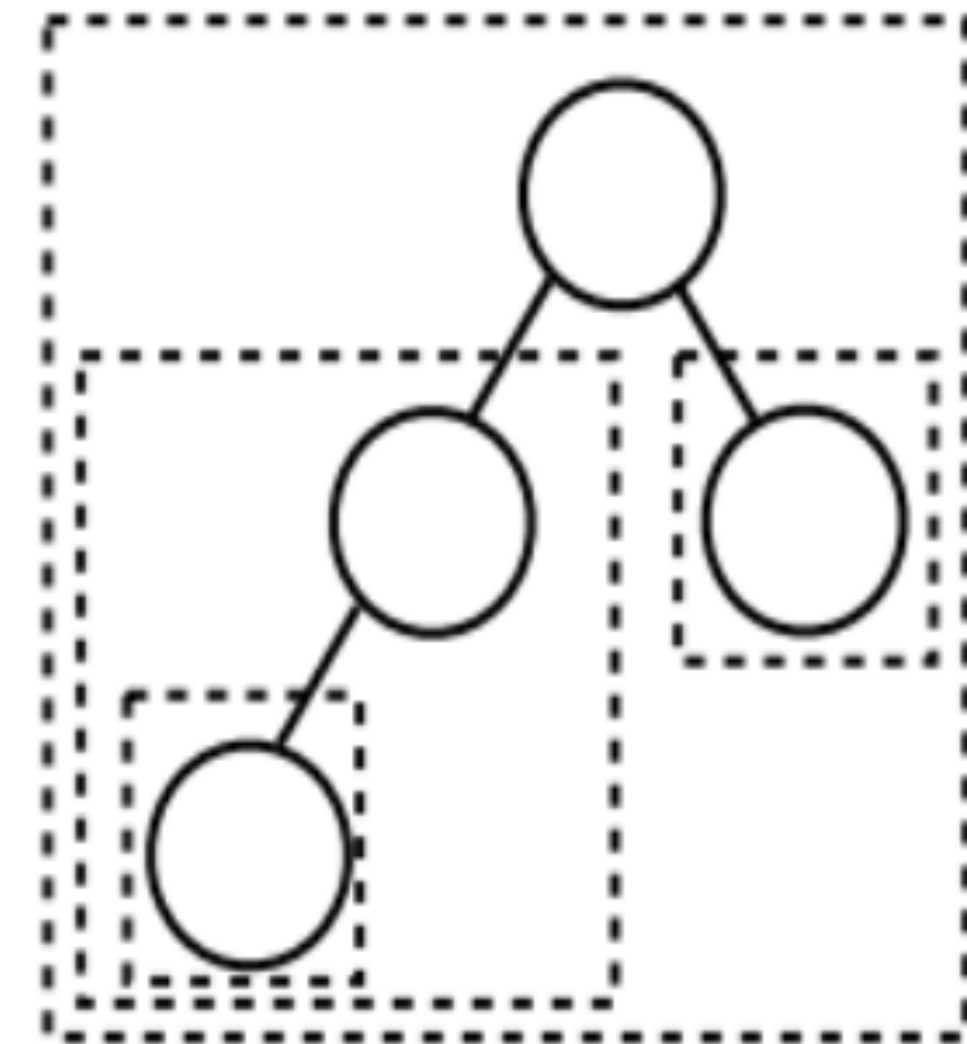


Lecture 22

Binary Search Tree

Binary tree: a node can have up to 2 children

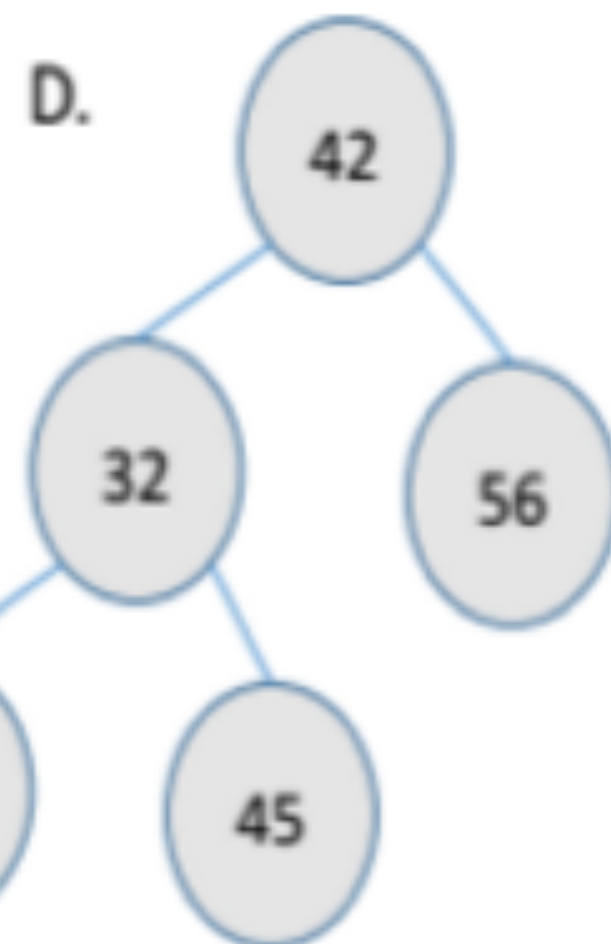
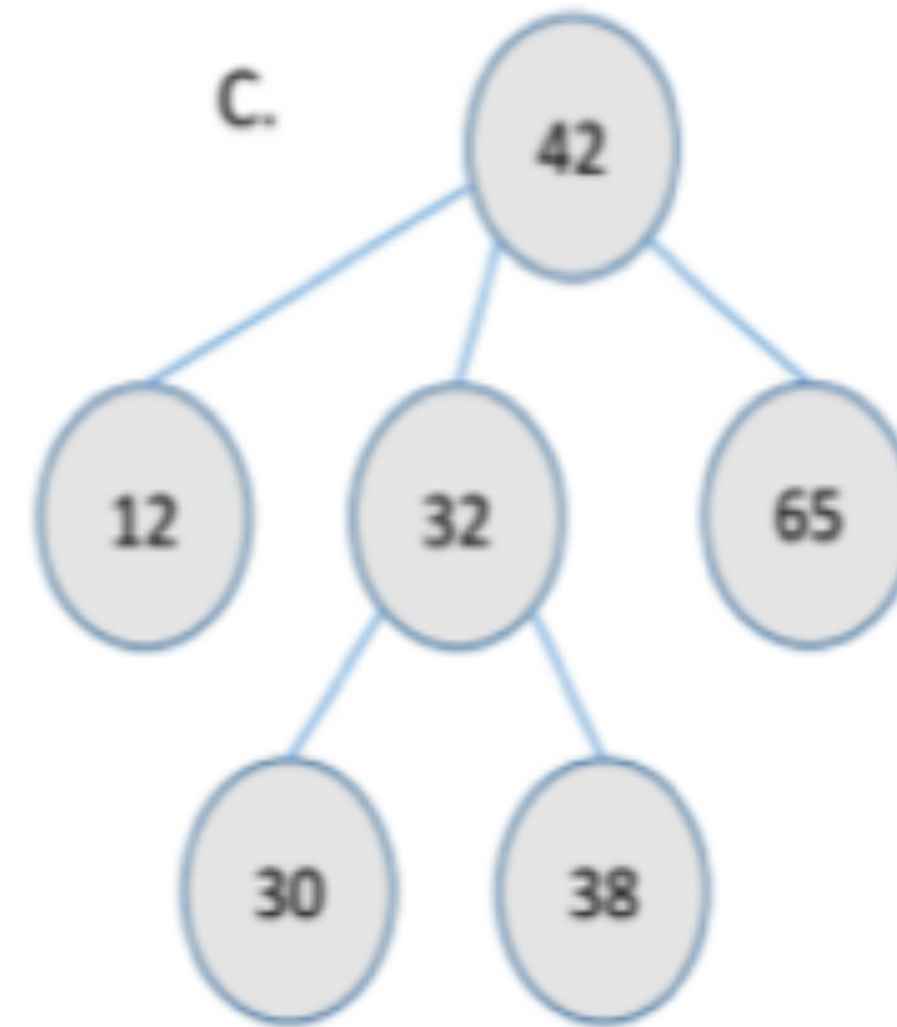
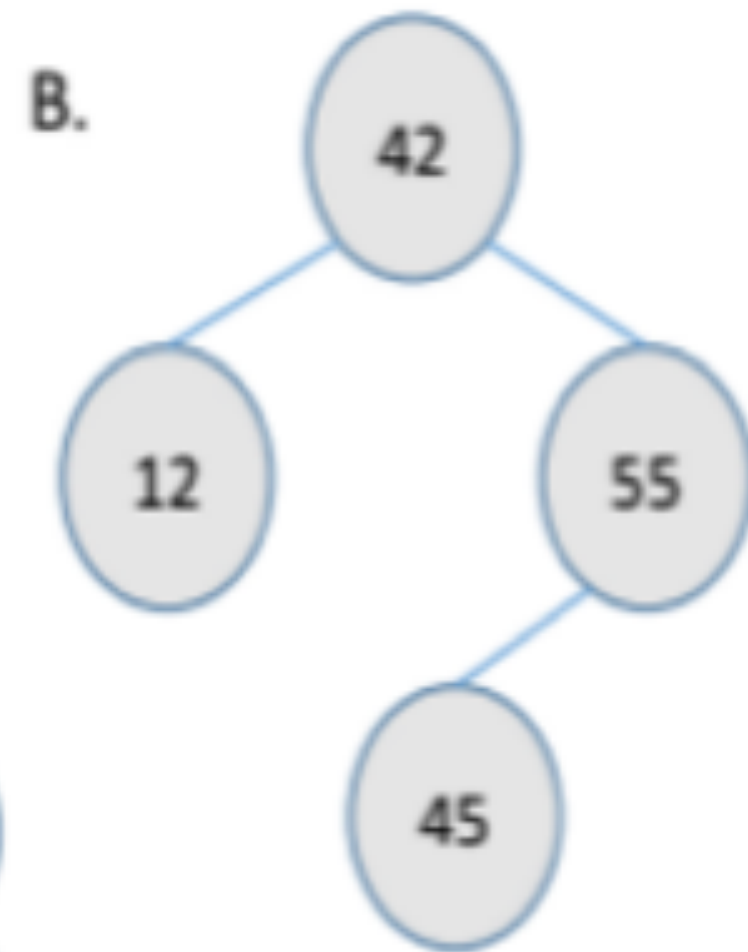
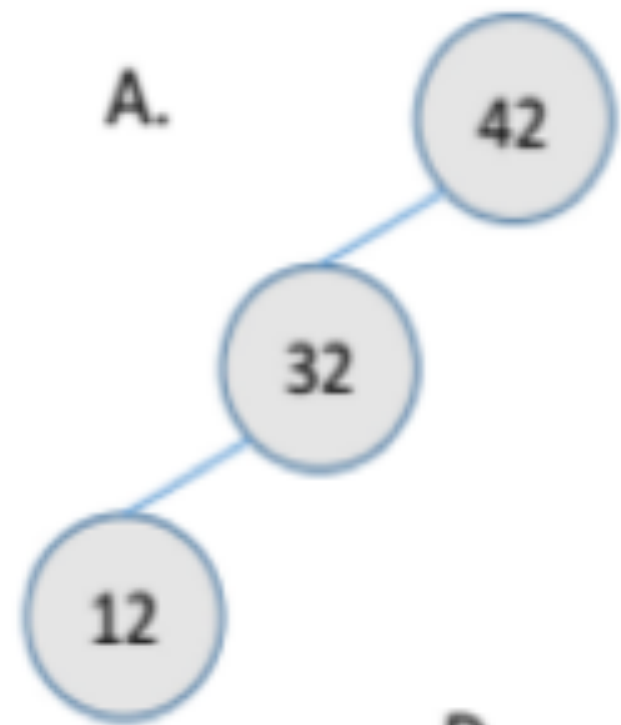
- Each node in a tree is the root of its own subtree
- The dotted boxes below show all possible subtrees



Binary search tree: binary tree w/ condition

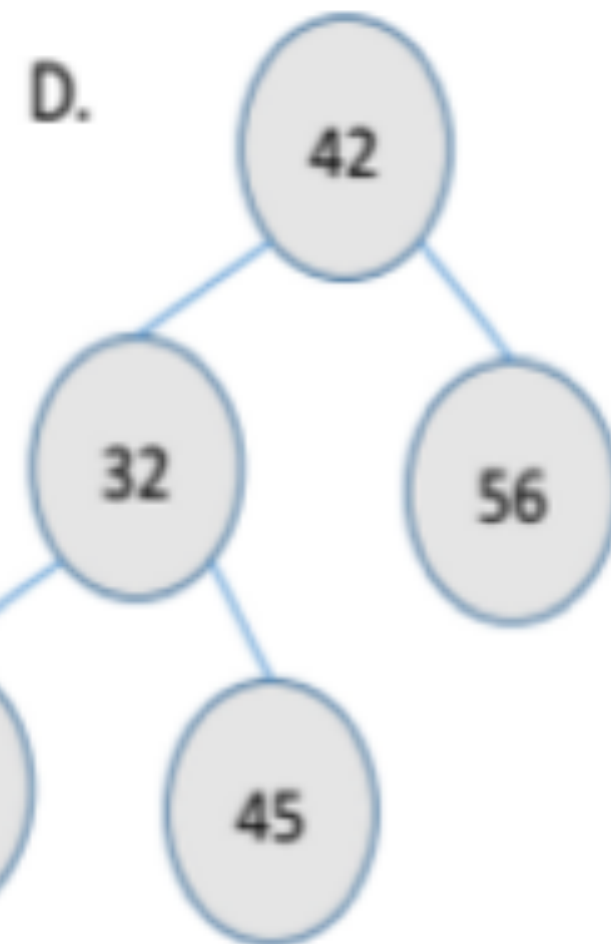
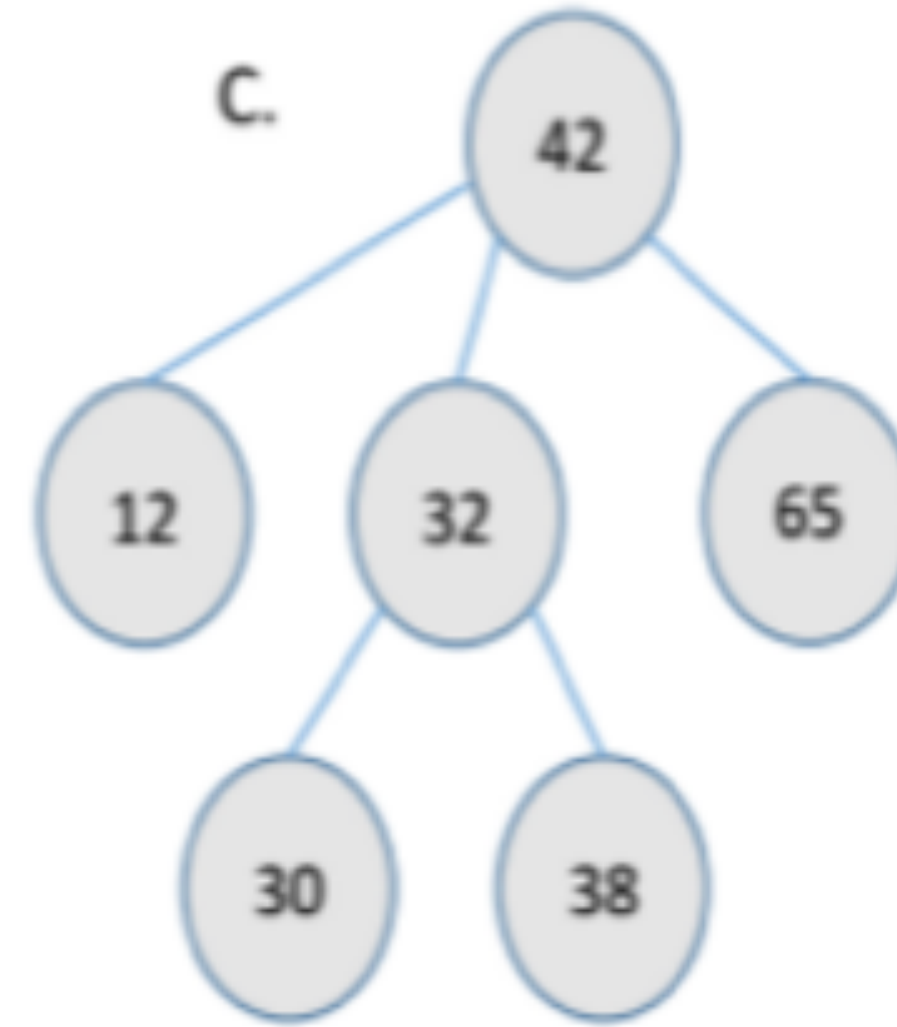
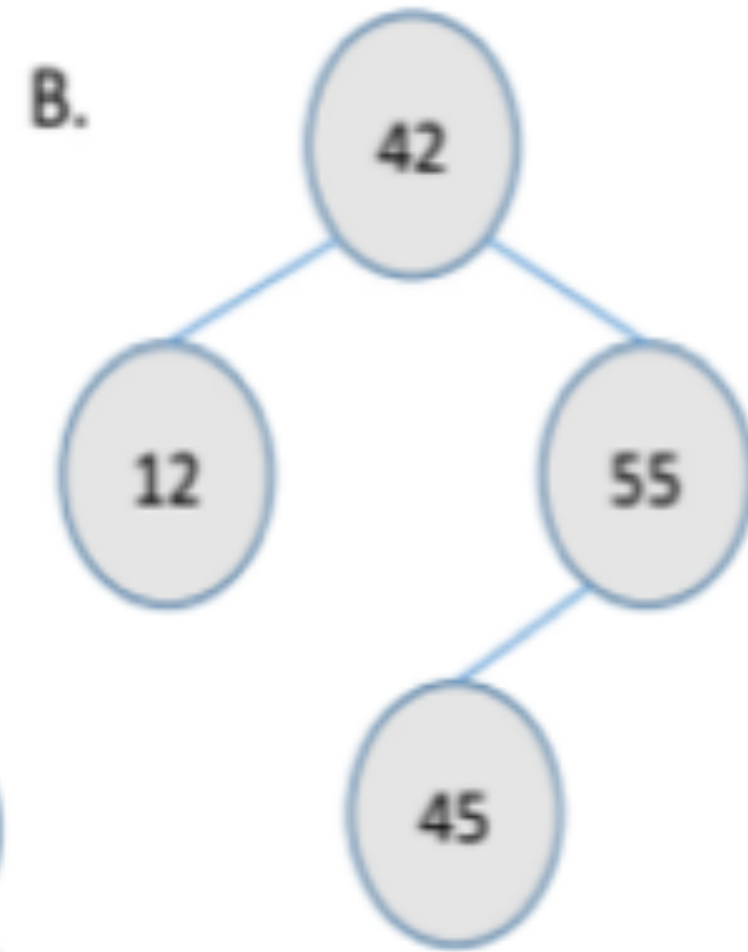
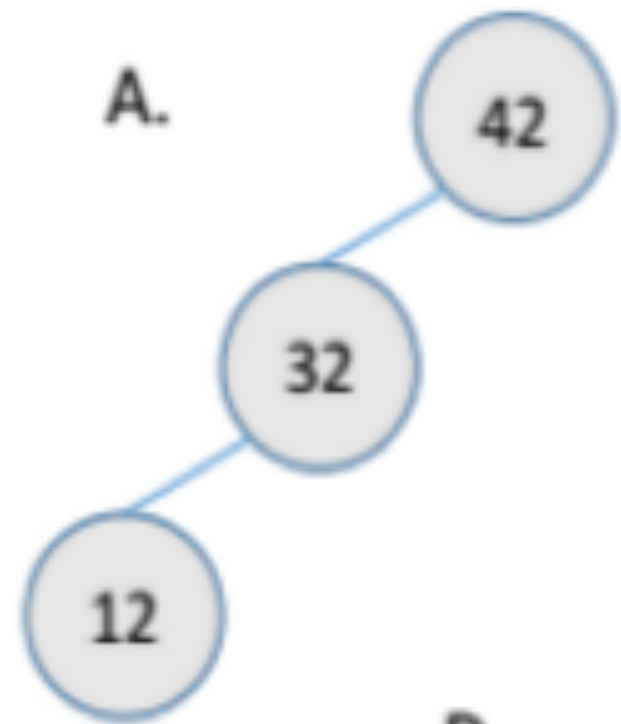
- Binary Search Tree is a node-based binary tree data structure which has the following properties:
 - The left subtree of a node contains only nodes with keys smaller than the node's key.
 - The right subtree of a node contains only nodes with keys greater than the node's key.

Which of the following is/are a binary tree?



E. More than one of these

Which of the following is/are a binary **search** tree?

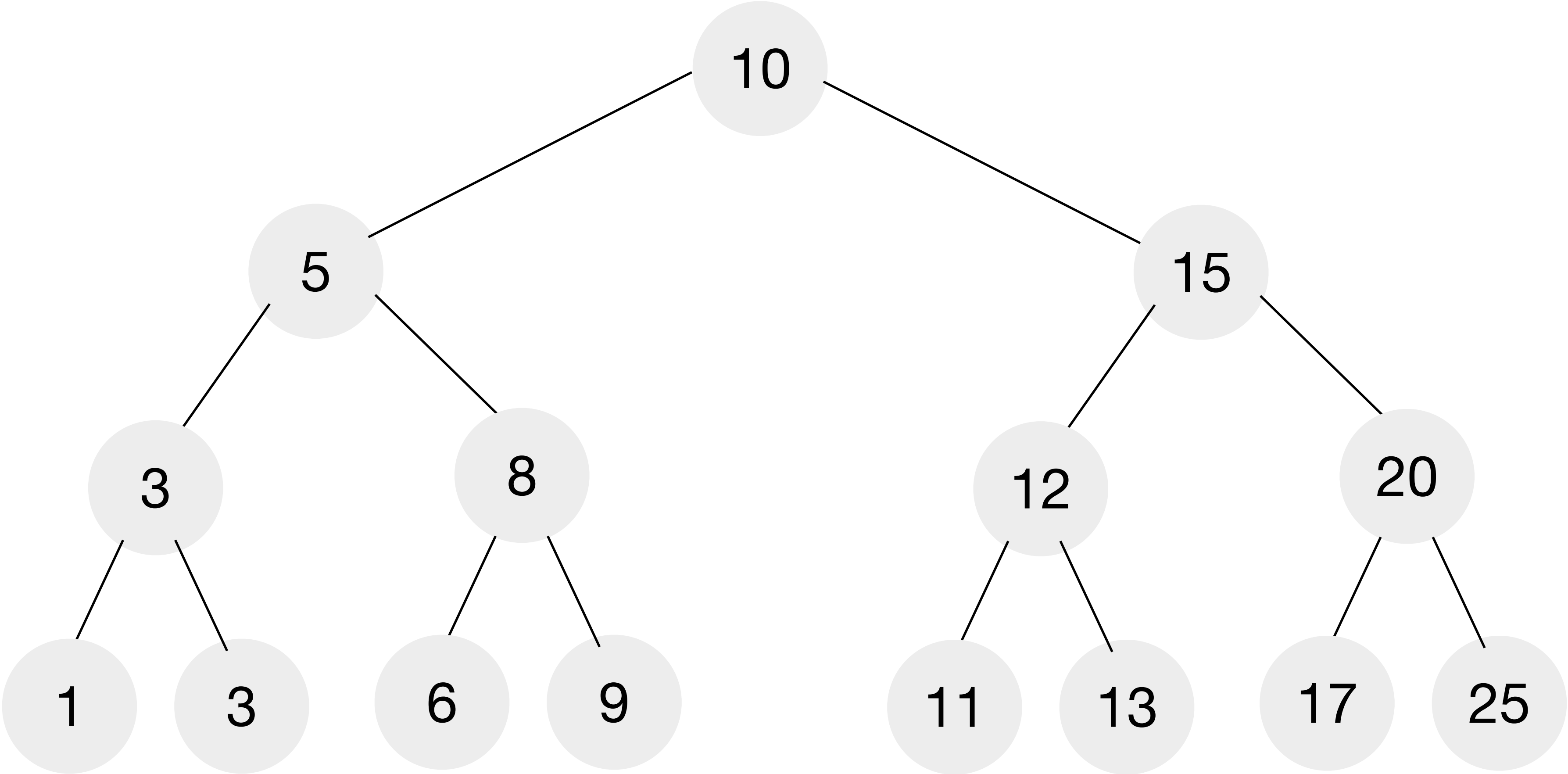


E. More than one of these

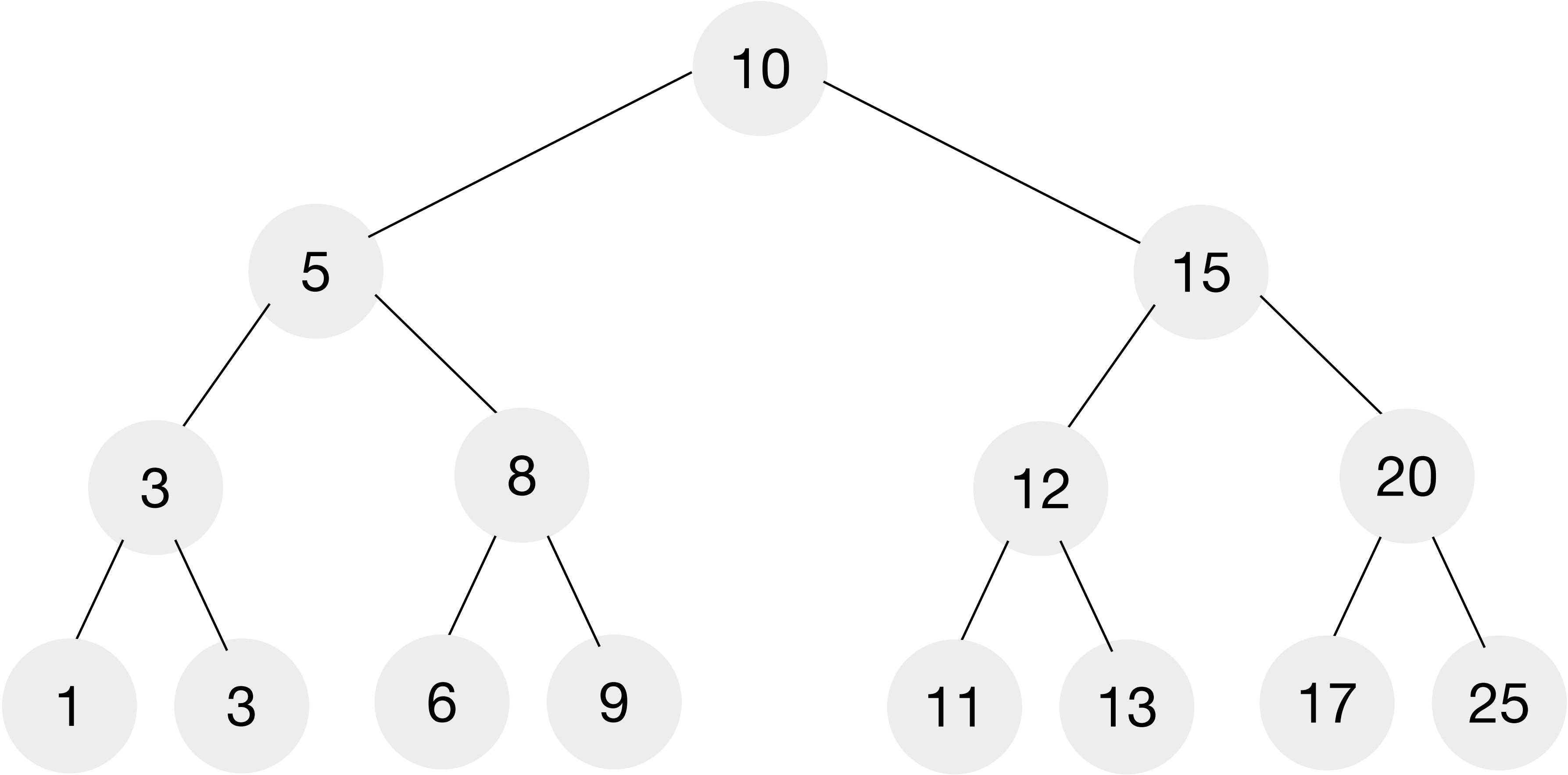
Why use Binary Search Trees?

- Binary trees are useful for storing data in an organized manner so that it can be quickly retrieved, inserted, updated, and deleted.
- This arrangement of nodes allows each comparison to skip about half of the rest of the tree, so each operation as a whole is very fast

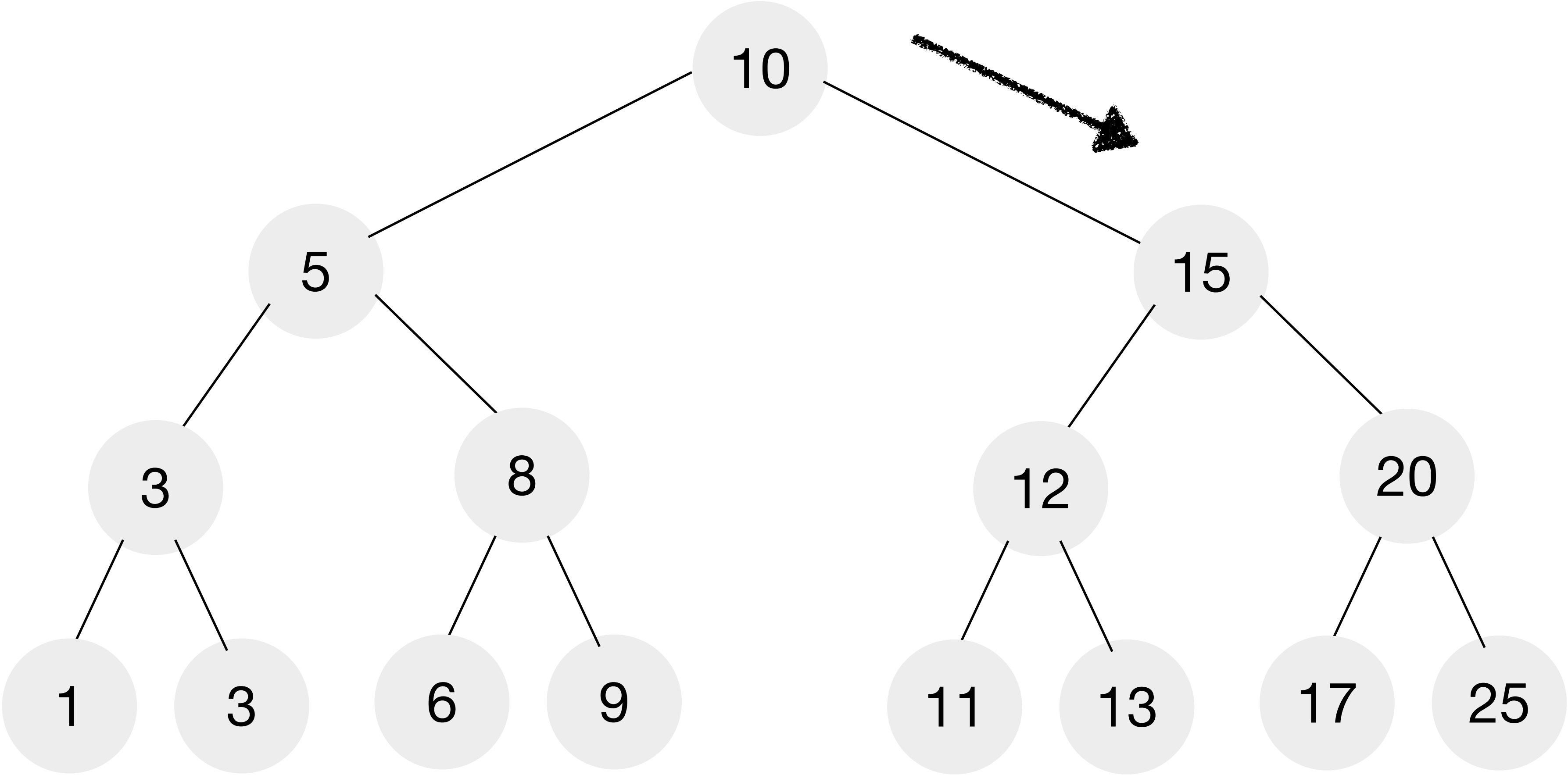
Searching BSTs



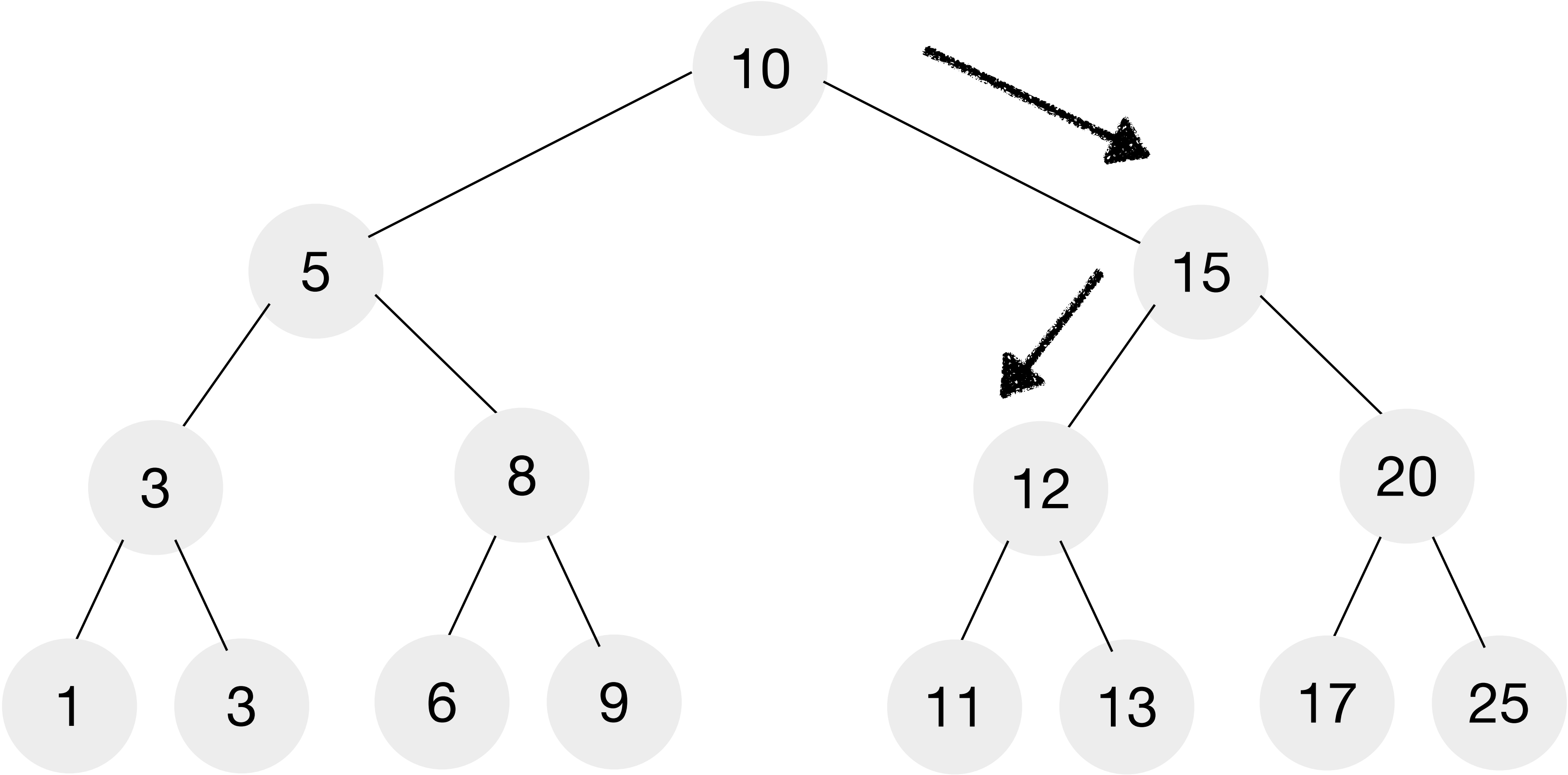
Searching BSTs: Find 11



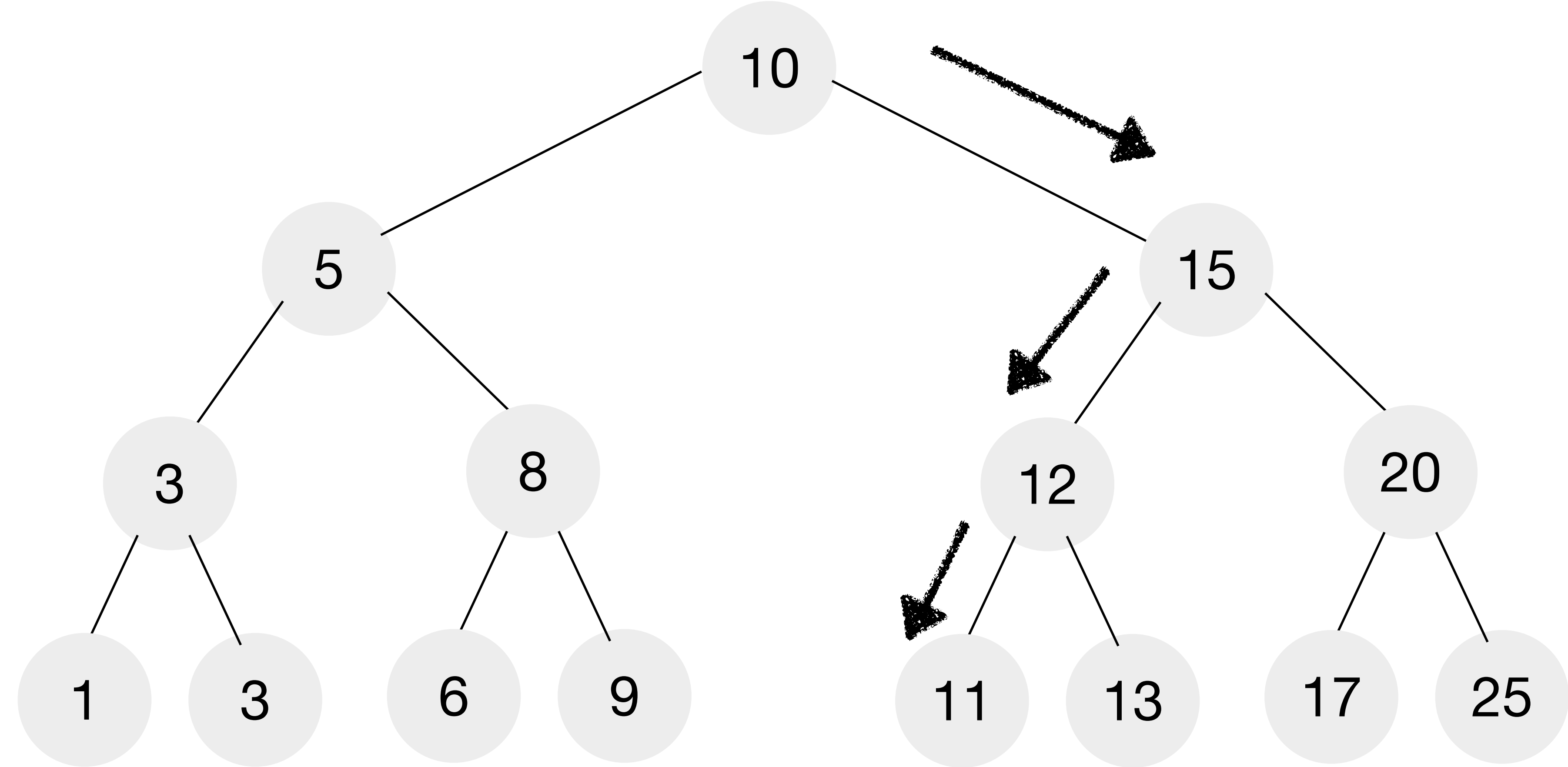
Searching BSTs: Find 11



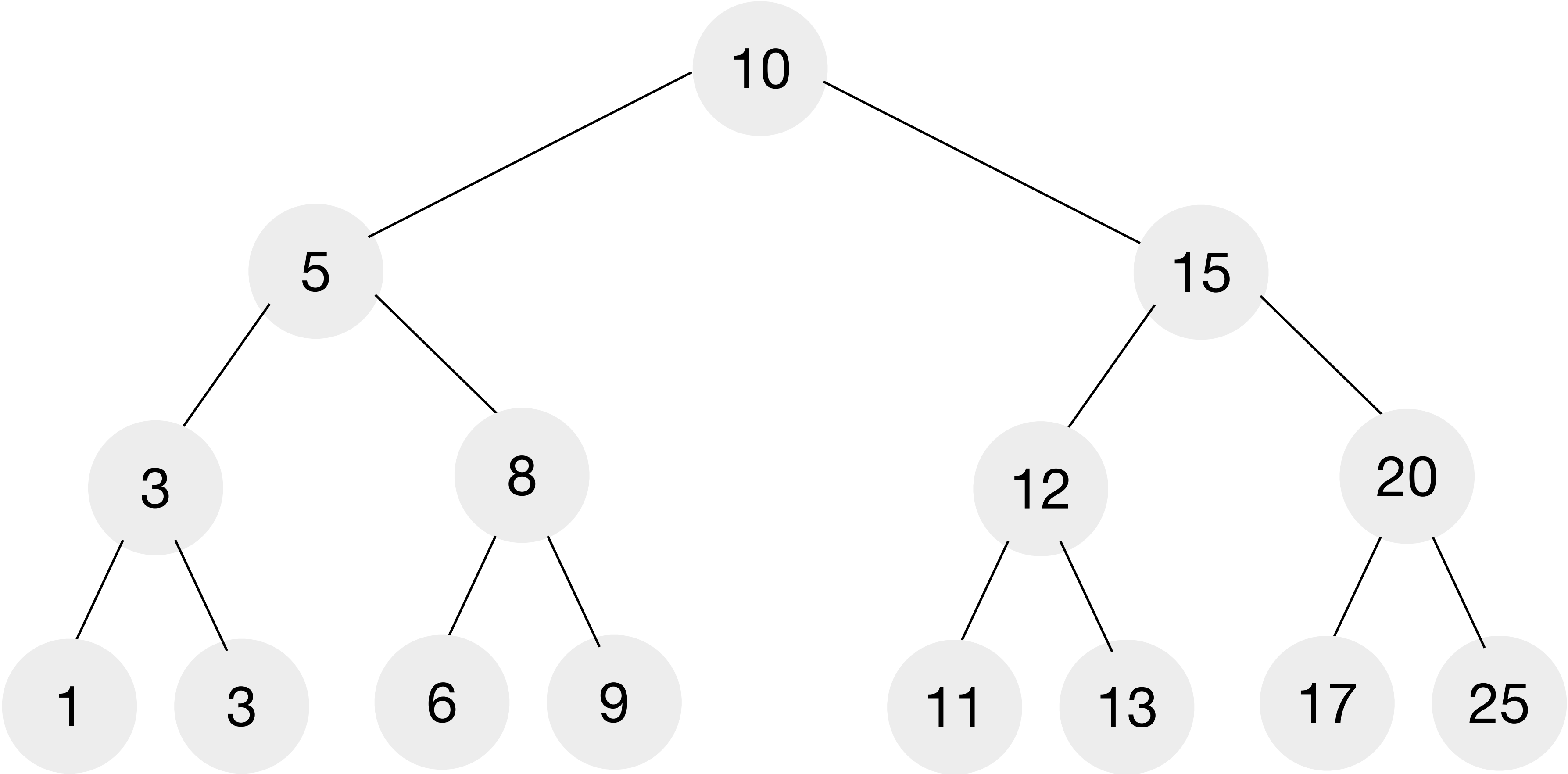
Searching BSTs: Find 11



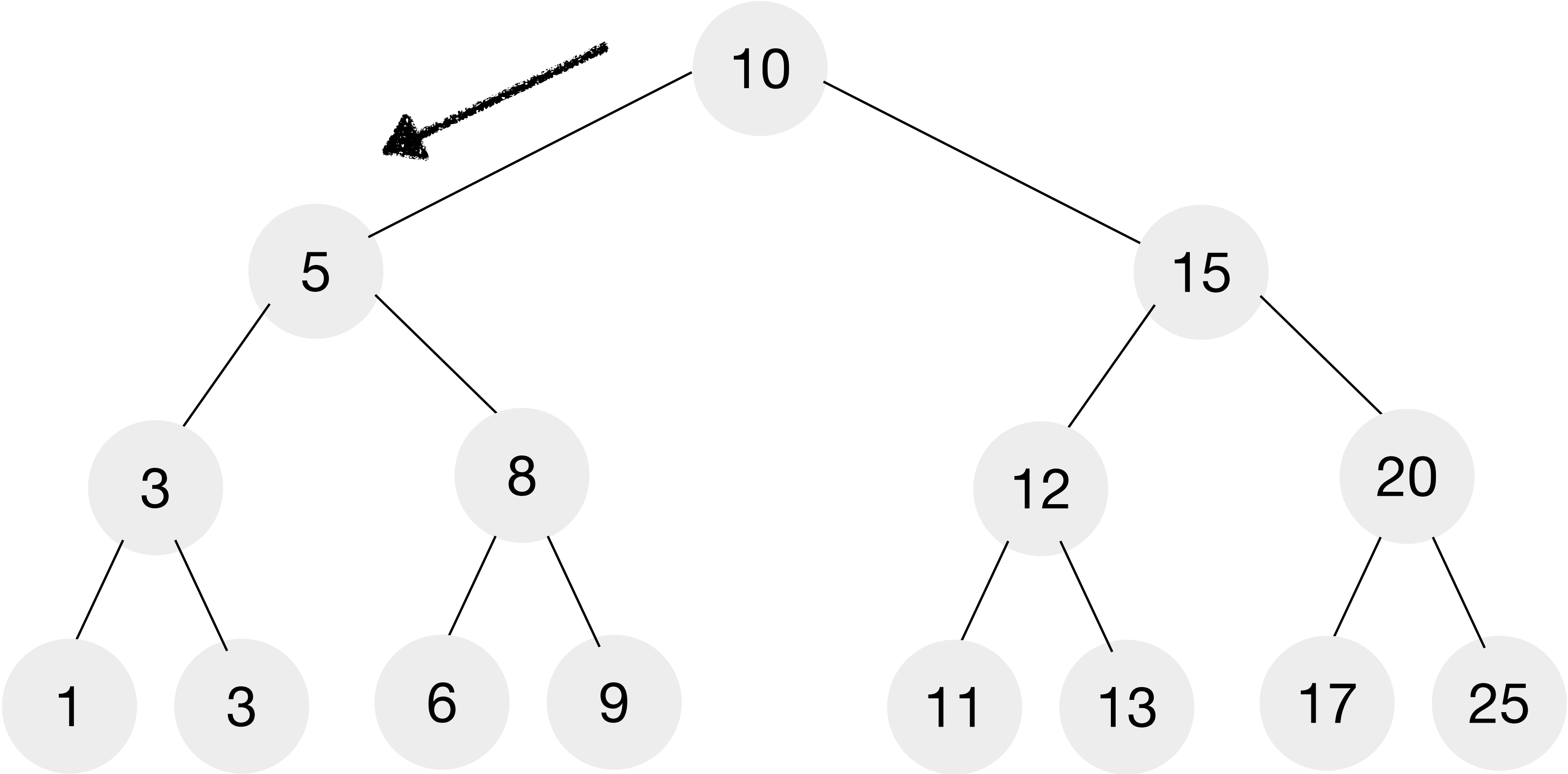
Searching BSTs: Find 11



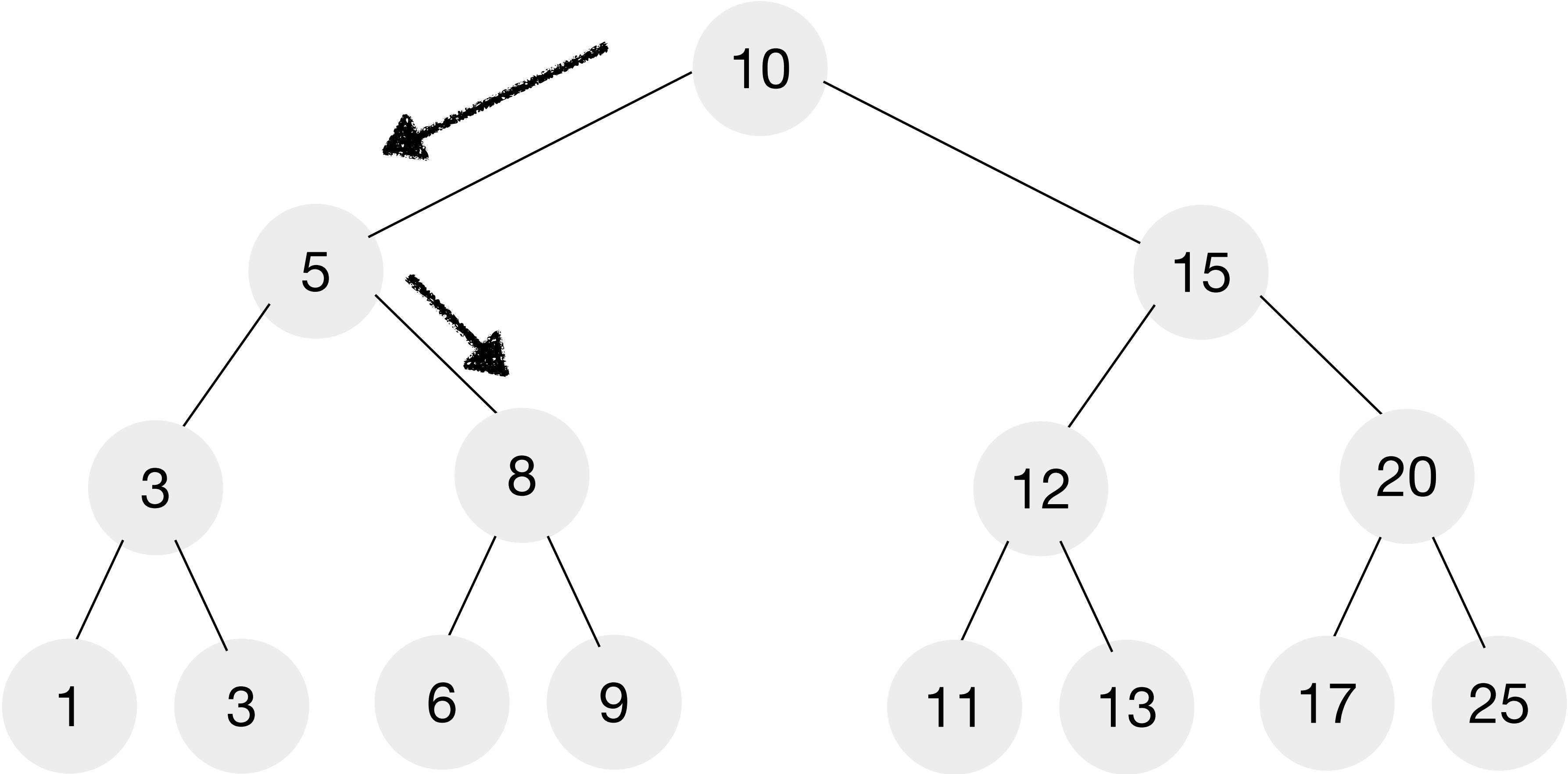
Searching BSTs: Find 6



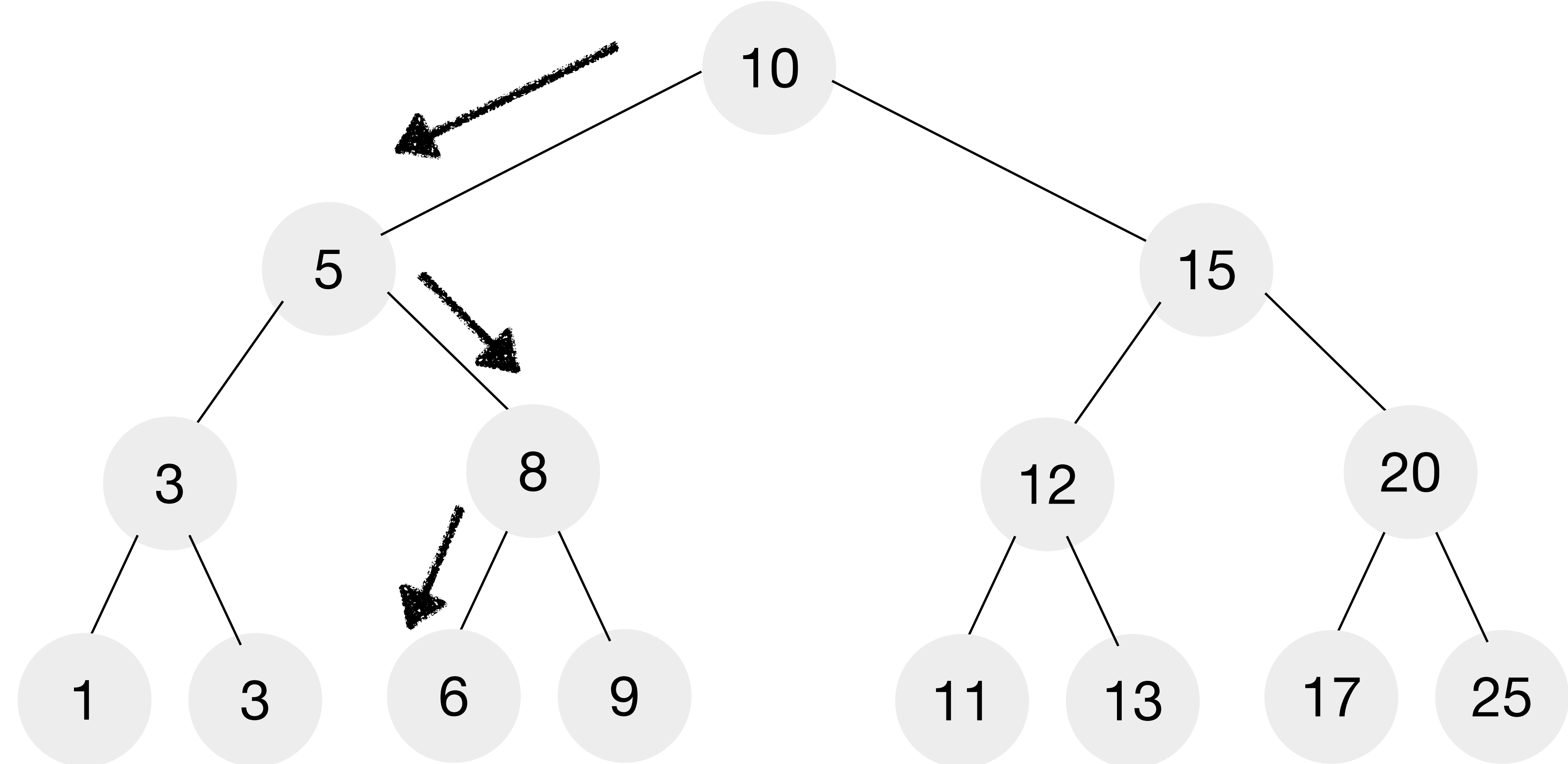
Searching BSTs: Find 6



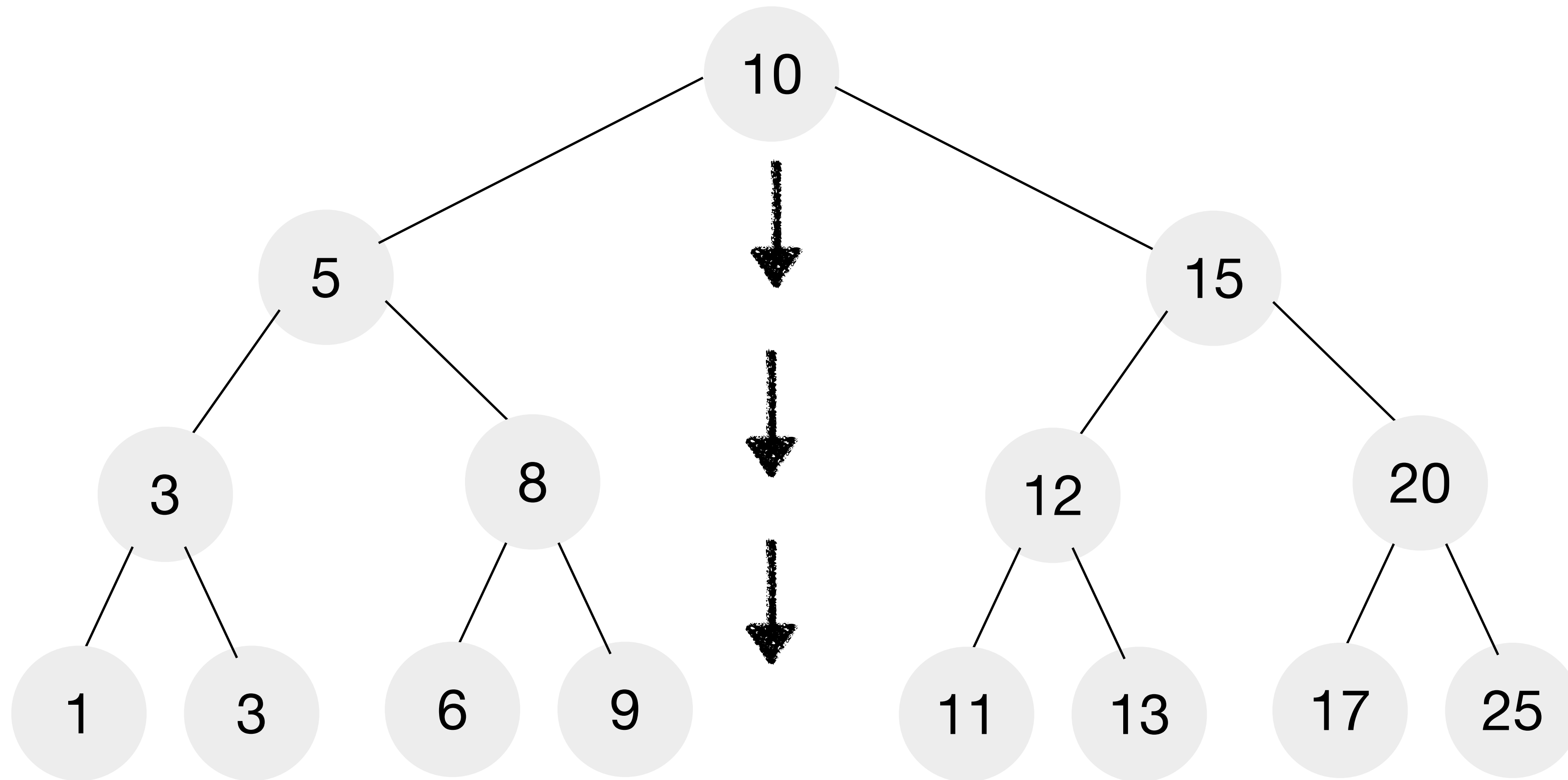
Searching BSTs: Find 6



Searching BSTs: Find 6



Searching BSTs, in general scales like the height of the tree



Binary Search Tree (BST) ADT

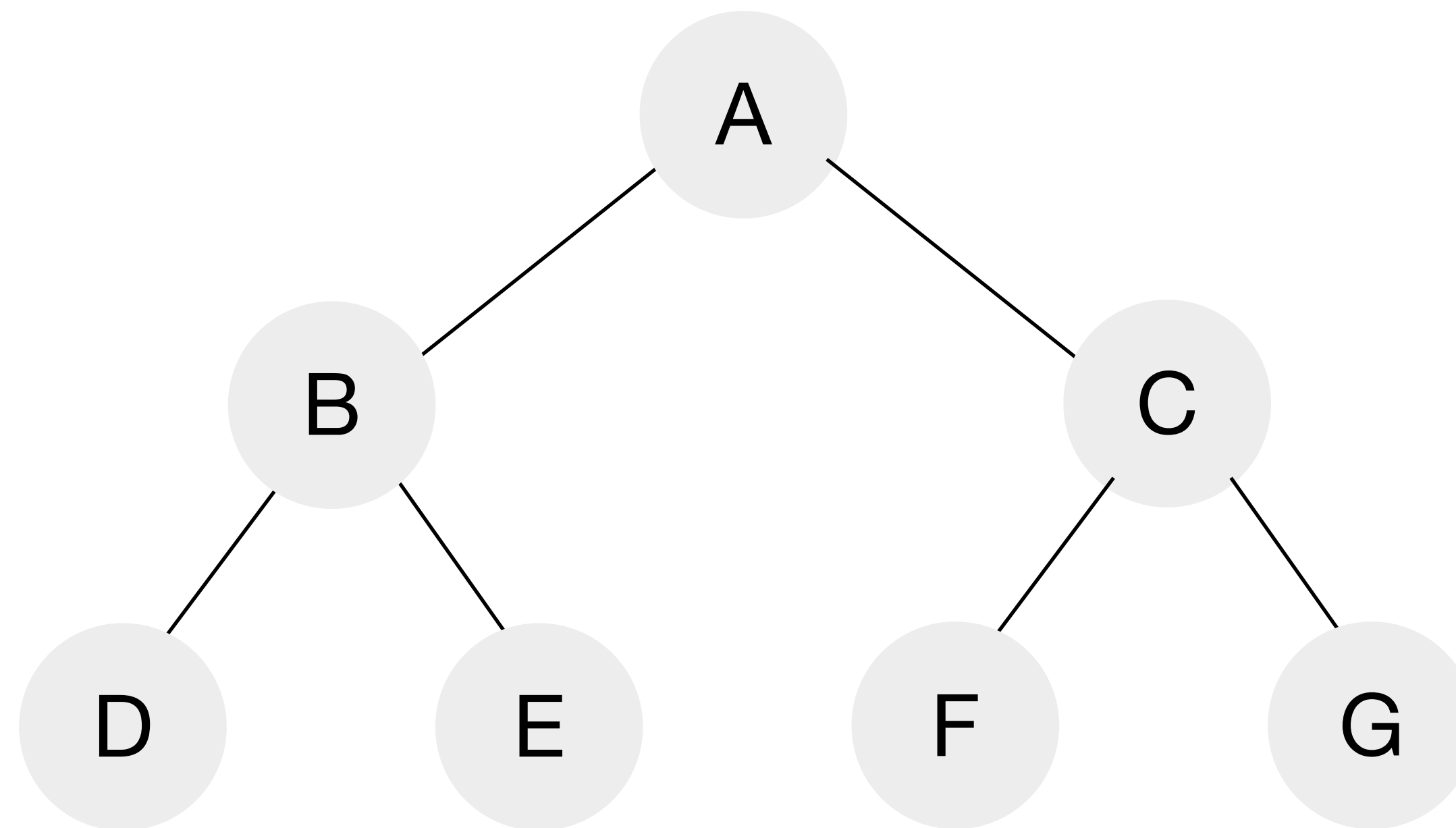
- Add(element)
- Find(element)
- Remove(element)
- ...
 - traverse()
 - findLargest (element)
 - removeLargest(element)

Binary Search Tree (BST) ADT

- Add(element)
- Find(element)
- Remove(element)
- ...
 - **traverse()**
 - findLargest (element)
 - removeLargest(element)

There are three ways to traverse BSTs

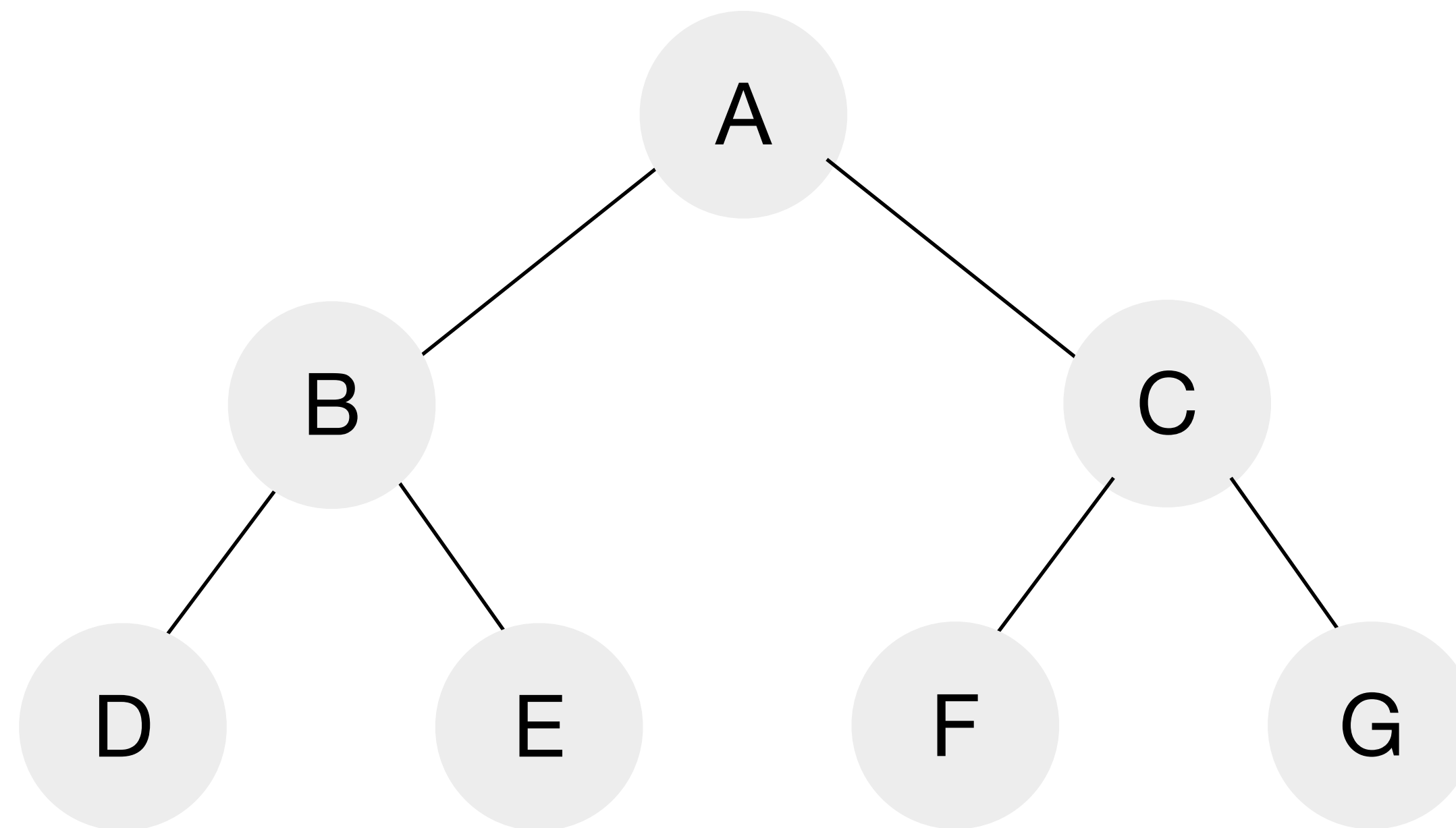
- Pre-order:
 - Root, left, right



A, B, D, E, C, F, G

There are three ways to traverse BSTs

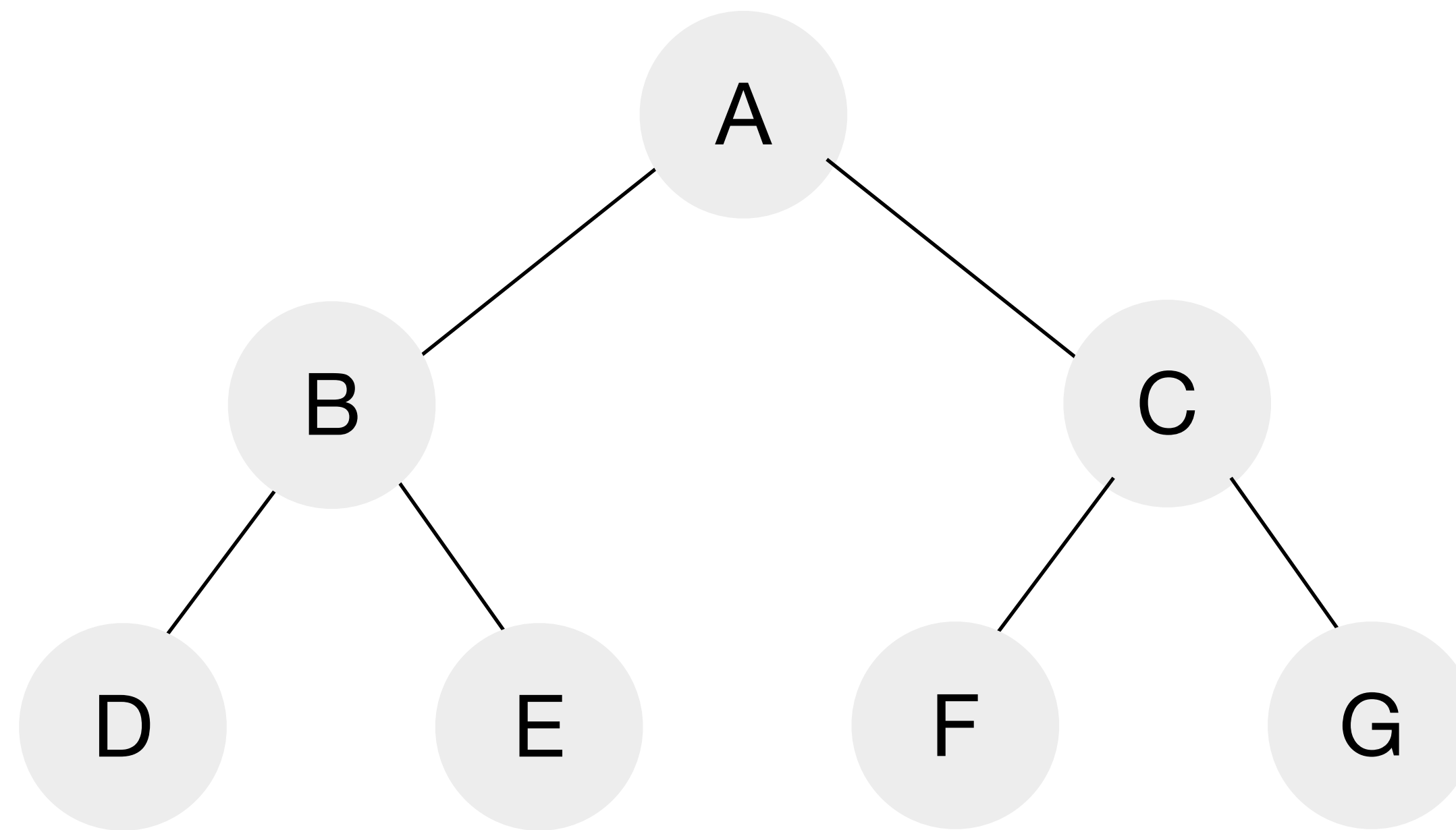
- Pre-order:
 - Root, left, right
- In-order
 - Left, root, right



D, B, E, A, F, C, G

There are three ways to traverse BSTs

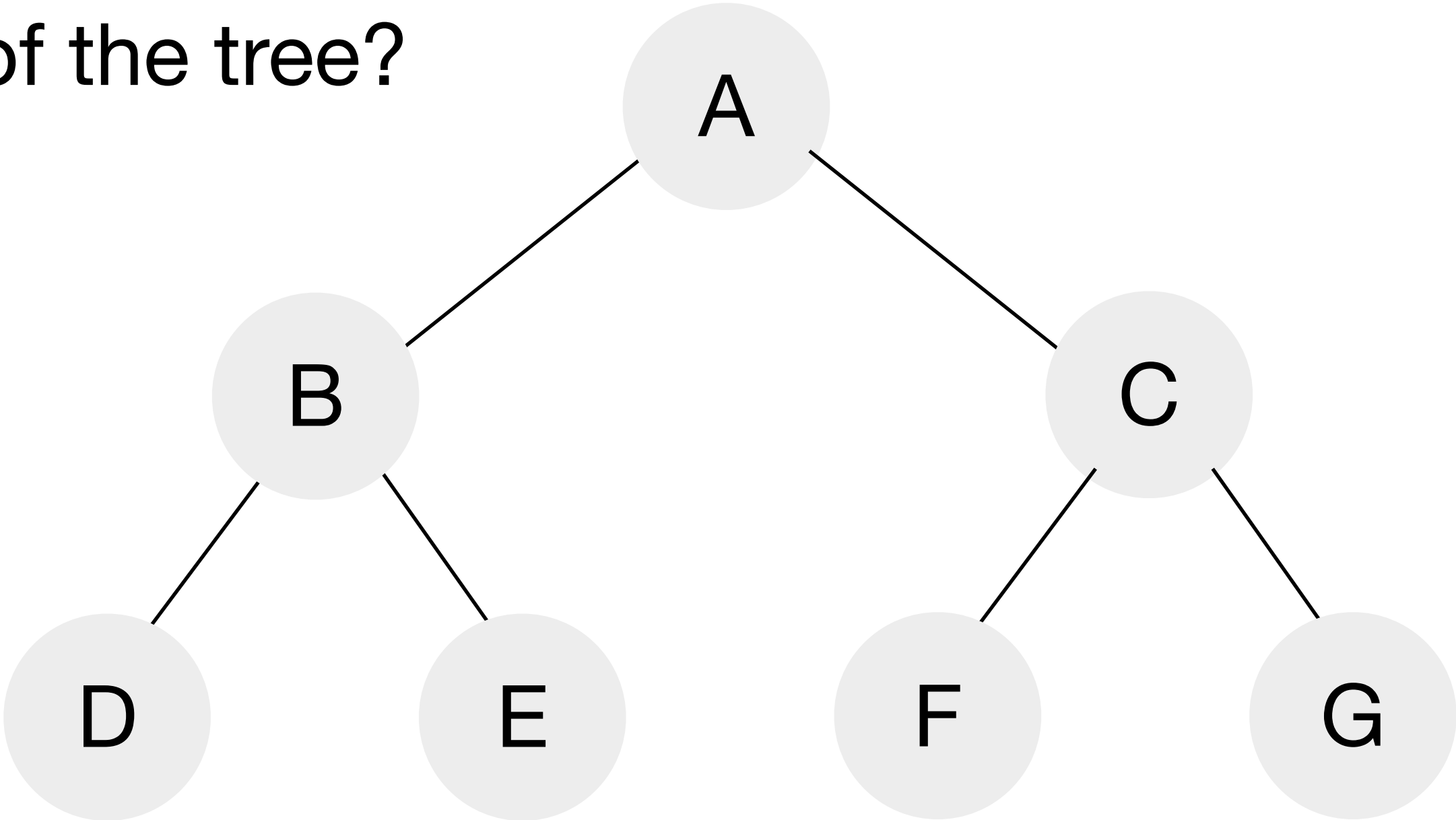
- Pre-order:
 - Root, left, right
- In-order
 - Left, root, right
- Post-order:
 - Left, right, root



D, E, B, F, G, C, A

In what order the code below visits the nodes of the tree?

```
def inorder(node):  
    if node.left is not None:  
        inorder(node.left)  
  
    visit this node  
  
    if node.right is not None:  
        inorder(node.right)
```

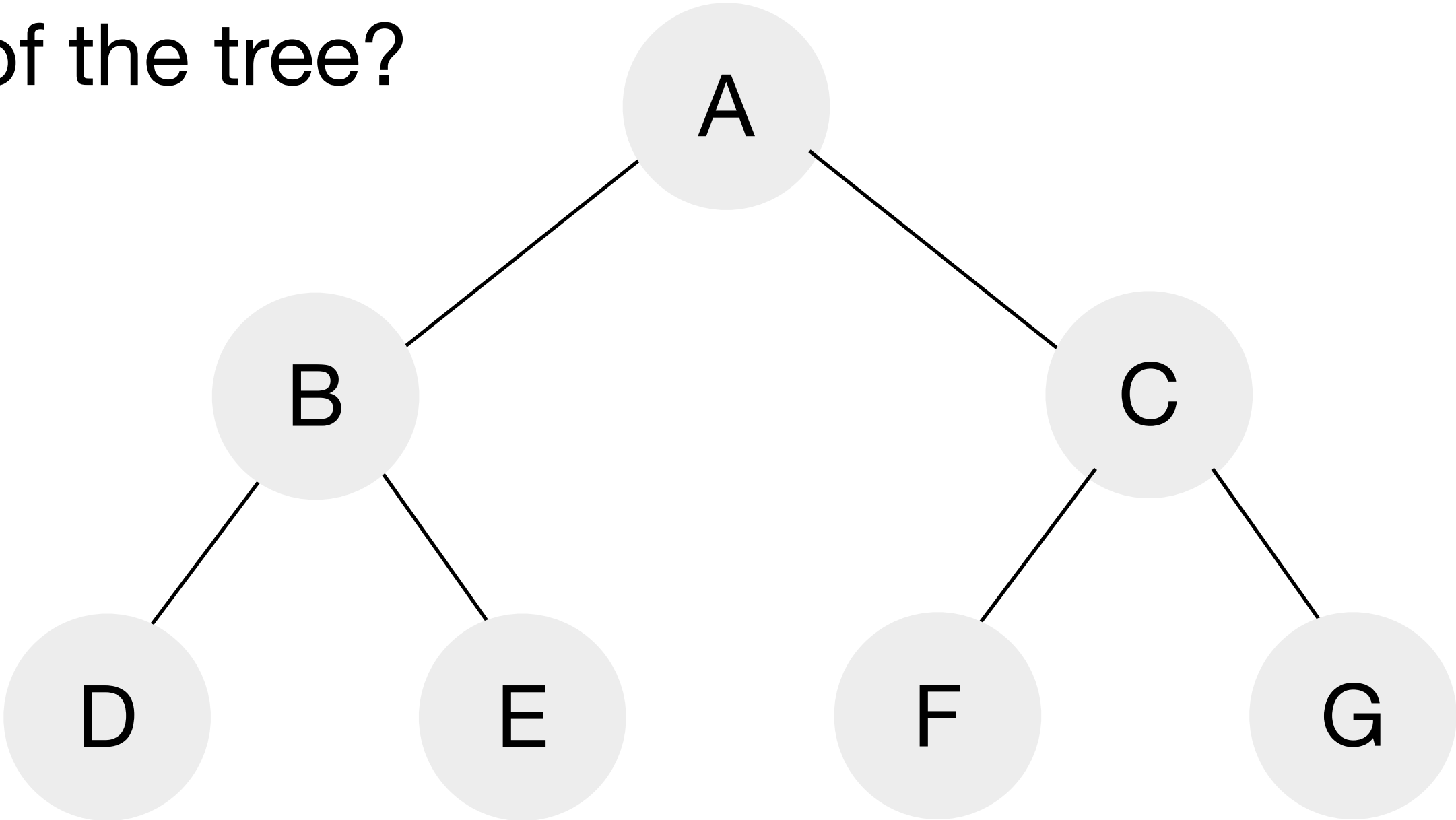


A	D B E A F C G
B	A B D E C F G
C	A B C D E F G
E	Other/none/more

class[Buzz]: bst_order1

In what order the code below visits the nodes of the tree?

```
def postorder(node):  
    if node.left is not None:  
        postorder(node.left)  
  
    if node.right is not None:  
        postorder(node.right)  
  
    visit this node
```



A	D B E A F C G
B	A B D E C F G
C	D E B F G C A
D	Other/none/more

class[Buzz]: bst_order

BST Class

```
class BSTNode:
    def __init__(self, value):
        <COMPLETE THE BODY>
```

```
class BST:
    def __init__(self):
        self.root = None

    def add(self, value):...

    ... other methods...
```

add(self, value) needs a helper function

- how can we add a node to a BST?

add(self, value) needs a helper function

- how can we add a node to a BST? You need to use recursion

```
def add(self, value):  
    if self.root is not None:  
        print ("the tree is not empty. finding an empty spot to add")  
        self.addHelper(self.root, value)  
    else:  
        print ("the tree is empty. adding the node to the head")  
        self.root = BSTNode(value)
```

Complete addHelper().

```
def add(self, value):  
    if self.root is not None:  
        self.addHelper(self.root, value)  
    else:  
        self.root = BSTNode(value)  
  
def addHelper(self, node, value):
```

Complete addHelper().

```
def add(self, value):
    if self.root is not None:
        self.addHelper(self.root, value)
    else:
        self.root = BSTNode(value)

def addHelper(self, node, value):
    if node.value < value:
        if node.right is None:
            node.right = BSTNode(value)
        else:
            self.addHelper(node.right, value)
    else:
        if node.left is None:
            node.left = BSTNode(value)
        else:
            self.addHelper(node.left, value)
```

Can a BST have duplicate keys?

- Yes, but you have to be consistent with the rule
 - Ex: duplicates always go to the right
- Better alternative is to keep them inside a single node as a linked list

remove(element) is a bit more complicated

- There are three cases
 - The removing node is a leaf-node
 - The removing node has 1 child (left/right)
 - The removing node has 2 children

